

Kong Ingress Controller — Rate Limiting, Auth & Caching

Nexlify Technologies | Gateway: kong-prod, kong-nonprod | DB-less mode

1. Kong Deployment Model

Kong Gateway is deployed in DB-less mode on both EKS clusters, managed by the Kong Ingress Controller (KIC). Configuration is entirely declarative via Kubernetes CRDs (KongPlugin, KongIngress, KongConsumer, Ingress). The controller watches these resources and translates them into Kong's in-memory configuration. Admin API is disabled in production; configuration changes flow only through GitOps. Kong pods run behind the external NGINX Ingress for public APIs and can also be reached via the internal NGINX for private APIs. Horizontal scaling is driven by HPA on CPU and custom request-rate metrics.

2. Rate Limiting Policies

Rate limiting is implemented with the Kong rate-limiting plugin (or rate-limiting-advanced where available). Policies are attached either globally, per-route, or per-consumer. Standard tiers used by Nexlify: Free-tier consumers are limited to 60 requests/minute; Standard tier 600 requests/minute; Premium tier 6000 requests/minute. Limits are enforced by consumer credential (API key or JWT subject). Redis is used as the shared counter store for distributed rate limiting across Kong replicas; the Redis instance lives in the same cluster (see non-prod Redis guidance for the dev pattern). When a limit is exceeded, Kong returns HTTP 429 with a Retry-After header. Burst settings allow short spikes above the sustained rate. Platform teams monitor the `kong_http_requests_total` and rate-limit rejection counters in Grafana.

3. Authentication & Authorization

Supported authentication plugins: key-auth (API keys stored as KongConsumer credentials), jwt (RS256 with public keys from the corporate JWKS endpoint), and oidc (for browser-based flows). Authorization is performed either by the ACL plugin (group membership) or by requesting the upstream service to validate fine-grained claims. Mutual TLS between Kong and upstream is optional and enabled for high-sensitivity routes via the service mesh or Kong's own mTLS plugin. Consumer objects are managed via GitOps; self-service key issuance is not yet available. Failed authentication attempts are logged and counted toward a separate anomaly metric that feeds the SIEM.

4. Response Caching

The Kong proxy-cache plugin is enabled on selected read-heavy routes (product catalogue, configuration endpoints). Cache key is derived from the request path, query string, and selected headers. TTL defaults to 60 seconds and can be overridden per-route. Cache storage is in-memory for low-latency routes; for larger working sets Redis is used as the cache backend. Cache hit ratio is exported as a Prometheus metric. Cache invalidation is performed by sending a PURGE request or by waiting for TTL expiry; application teams must design for eventual consistency when caching is active.

5. Operational Checklist

- Verify Kong pods are Ready and the Ingress Controller has synced all resources.
- Confirm rate-limit Redis connectivity from Kong pods.
- Review consumer and credential objects after any access-control change.
- Check cache hit ratio and 429 rates after traffic spikes.
- Ensure new routes have an explicit authentication plugin unless the route is intentionally public.
- After controller upgrade, validate that custom plugins still load.

Owner: API Platform | Slack: #kong-gateway | Rev 2.8 | 2026-06-22